AI Security Auditor — 감사 파이프라인 개선 기록

> 포트폴리오용 개선 기록. 항목별로 **문제 → 원인 → 해결 → 검증** 순서로 정리한다. 작업 목록: `Todo.md` · 기간: 2026-11-06 ~
> 기준선: 백엔드 테스트 277개 통과 · A/B 실측(`reports/eval_6e3da199.md`) B 구성 F1 0.93, 감사 1건 84.6초 / 약 86k 토큰

요약

영역	핵심 개선	대표 수치
정확성·안정성	400줄 넘는 파일의 근거 검증 실패, 락 만료, Kafka 30분 상한, 재개 시 예산 초기화, ReAct 시간·토큰 한도	버그 6건 수정, 각각 회귀 테스트
속도	ReAct·probe 병렬화, MCP 세션 재사용, 이벤트·span 묶음 기록, 워커 동시 처리	단계별 **-63% ~ -89%** (2.7~9.4배)
탐지 범위	카테고리 확장, 여러 줄 매칭, Spring 규칙 10개, Python 규칙 10개(AST 포함), 내용 기반 위험 파일 선택, 파일 간 흐름, Python 에이전트 도구 탐색	규칙 17 → 37개 · fixture 탐지 0/53 → **53/53, 오탐 0** · Python 에이전트 도구 1/7 → 7/7
외부 도구	Bandit·Semgrep·SCA(로컬 권고 DB + 선택적 OSV)	취약 의존성 0/6 → **6/6**
평가 체계	Python 정답셋, 대상별 평가 도구, 채점 버그 2건, 비용 산출, 보고서 소요 시간	정적 기준선 F1 Spring 0.837 → **0.889**, Python 탐지 0 → **0.857**
리뷰 반영 (6장)	락·lease 원자화와 소유자 확인, lease 상실 시 처리 취소, 병렬 LLM 예산 예약, 병렬 실패 시 정리	병렬 8회 호출 토큰 상한 초과 **2배 → 0**, 수정 전 코드에서 실패하는 회귀 테스트 11개

- 테스트: 277개 → 310개 → **325개** 통과 (`make test`), ruff·mypy 통과
- 미완료: 실제 LLM 재측정(EV5) — `.env` 의 OpenAI 키가 만료되어 있음(5-5)

---

1. 정확성·안정성

1-1. 400줄 넘는 파일의 근거 검증이 항상 실패하던 문제

- **문제**: LLM·정적 규칙이 찾은 취약점이라도 파일의 401번째 줄 이후에 있으면 근거 검증(H4 가드레일)에서 항상 "파일·라인 불일치"로 판정됐다. confidence가 0.3 깎이고 `NEEDS_REVIEW` 로 떨어졌다.
- **원인**: 근거 검증 노드가 `read_repo_file` 을 라인 범위 없이 호출했다. 이 도구는 한 번에 최대 400줄만 돌려주기 때문에 나머지 줄은 검증 대상에서 빠졌고, 라인 보정(`relocate_evidence`)도 앞 400줄 안에서만 찾았다. 데모 대상은 파일이 짧아 드러나지 않았고, 실제 업로드 프로젝트에서만 생기는 문제였다.
- **해결**: `read_full_file()` 을 추가해 `total_lines` 까지 400줄씩 이어 읽고 파일별로 캐시한다(최대 20,000줄). MCP 도구의 400줄 제한(컨텍스트 보호용)은 그대로 둔다.
- **검증**: 900줄 파일의 450번째 줄 근거 통과(호출 3회), 600번째 줄로 라인 보정되는 단위 테스트 추가.
- 파일: `backend/core/graph/nodes/code_evidence_check.py`

1-2. 감사가 30분을 넘으면 대상 락이 풀리던 문제

- **문제**: 같은 대상을 두 감사가 동시에 건드리지 못하게 Redis 락(`SET NX EX 1800`)을 거는데, 연장 장치가 없었다. 샌드박스 준비(최대 1200초)에 ReAct 조사와 probe 실행 시간이 더해지면 감사 도중 락이 만료되고, 같은 대상에 대한 두 번째 감사가 시작될 수 있었다.
- **해결**: `keep_locks_alive()` heartbeat 작업을 추가했다. 감사 그래프가 실행되는 동안 TTL의 1/3(10분)마다 락 TTL을 다시 30분으로 늘리고, 감사가 끝나거나 실패하면 cancel한다. **이 감사가 쥐고 있는 락만** 연장하고 다른 감사의 락은 건드리지 않는다. Redis 오류는 경고 로그만 남기고 다음 주기에 다시 시도한다.
- **검증**: 내 락은 연장되고 다른 감사의 락은 그대로인지 확인하는 단위 테스트, 기존 워커·그래프 통합 테스트 통과.
- 파일: `backend/core/redis/locks.py`, `backend/workers/audit.py`

1-3. Kafka 처리 시간이 30분으로 묶이던 문제

- **문제**: audit-worker는 메시지를 받은 뒤 감사가 끝날 때까지 `getmany()`를 호출하지 않았다. 감사가 `max_poll_interval_ms`(30분)를 넘으면 Kafka가 이 컨슈머를 죽은 것으로 보고 리밸런스하고 메시지를 다시 전달한다. 결국 30분이 감사 1건의 사실상 최대 처리 시간이었다.
- **해결**: `consume_loop()` — 메시지 처리를 별도 task로 돌리고, 처리 중인 파티션은 `pause()` 한 채 `getmany()`를 계속 호출한다. 바로 처리하지 못하는 레코드(같은 파티션이 처리 중이거나 리밸런스로 새로 할당된 경우 등)는 그 offset으로 `seek()`해 되감아서 유실되지 않게 한다. 처리가 끝난 파티션만 커밋하고 `resume()`한다. 처리 시간과 poll 주기가 분리되어 `max_poll_interval_ms`를 30분에서 5분으로 줄였고, 진짜로 멈춘 워커를 더 빨리 감지한다. (이 루프는 2-6에서 파티션별 동시 처리로 확장했다.)
- **검증**: 파티션 위치·pause·seek·commit을 흉내 내는 가짜 consumer로 "처리 중에도 poll 계속, 되감기, 전부 커밋, 끝나면 resume"을 확인하는 단위 테스트. 기존 재시도·DLQ 테스트 7개 통과.
- 파일: `backend/core/kafka/consumer.py`

1-4. 재개·재시도할 때 비용 예산이 0으로 돌아가던 문제

- **문제**: 감사당 토큰·비용 상한(H8, `max_tokens` 400k / `max_cost_usd` 2.0)을 두었지만, 워커가 재시작돼 체크포인트에서 이어가거나 Kafka가 재시도하면 사용량이 0부터 다시 계산됐다. 재시도가 반복되면 상한을 넘겨 비용을 쓸 수 있었다.
- **원인**: `Budget`은 실행 의존성(`Deps`) 안에만 있고 체크포인트에는 저장되지 않는다. 실행할 때마다 `Deps`를 새로 만들었다.
- **해결**: 체크포인트 state에 이미 쌓이는 `token_usage`(노드별 in/out/usd)를 이용해, 재개 시 `Budget.restore()`로 사용량을 복원한다. 새 저장소를 만들지 않고 기존 상태를 재사용했다.
- **검증**: 복원 후 사용량 합계와 예산 초과 판정을 확인하는 단위 테스트, 워커 통합 테스트 통과.
- 파일: `backend/harness/budget.py`, `backend/workers/audit.py`

1-5. ReAct 조사의 시간 제한이 실제로는 지켜지지 않던 문제

- **문제**: 파일당 ReAct 조사는 120초 제한(Harness Agent Loop)이 있었지만, 시간 확인을 **스텝 사이**에서만 했다. LLM 호출 1회가 60초 타임아웃 × SDK 재시도 2회 × 스키마 재요청 2회까지 늘어날 수 있어서, 한 스텝만으로 120초를 몇 배 넘길 수 있었다.
- **해결**: 매 LLM 호출(관찰 요약 포함)을 `asyncio.wait_for(남은 시간)`으로 감쌌다. 호출 도중 한도에 걸리면 `timeout`으로 루프를 끝내고, 마지막 판정(final) 요청에는 별도 유예 시간(`final_grace_s`, 30초)만 준다. final도 실패하면 해당 파일은 정적 규칙 판정으로 돌아가고 오류가 `errors`에 구조화되어 남는다.

- **검증**: 호출마다 5초씩 걸리는 가짜 LLM에 한도 0.3초를 주고, 2초 안에 `loop.limit(reason=timeout)` + `UPSTREAM_TIMEOUT`으로 끝나는지 확인하는 통합 테스트.
- 파일: `backend/core/graph/nodes/react_investigator.py`, `config/budget.yaml`

1-6. 파일당 토큰 예산이 LLM 토큰을 세지 않던 문제

- **문제**: 파일당 토큰 상한(`max_tokens_per_file`)이 도구 결과 텍스트 토큰만 셌다. 비용 대부분을 차지하는 LLM 입력·출력 토큰은 빠져 있어서 비용 제어 장치(H8) 구실을 못 했다.
- **해결**: 사용량 목록을 파일별로 따로 모으고(병렬화 이후에도 정확하게 셀 수 있도록), 스텝마다 그 파일의 LLM 입력+출력 토큰을 확인한다. `max_llm_tokens_per_file`(60k)을 넘으면 `llm_token_budget`으로 조사를 끝내고 판정한다. 도구 결과 상한(12k)은 그대로 둔다. 파일별 사용량은 실패하더라도 `finally`에서 감사 전체 사용량에 합쳐서 누락되지 않게 했다.
- **검증**: 호출당 150토큰인 가짜 LLM에 상한 300을 주고 2스텝 후 멈추는지, 사용량 합계(450)가 빠짐없이 기록되는지 확인하는 통합 테스트.
- 파일: `backend/core/graph/nodes/react_investigator.py`, `config/budget.yaml`

2. 속도 개선 (수치화)

> **측정 방법**: `backend/harness/eval/bench_speed.py` — LLM·MCP·Redis·DB·대상 에이전트의 응답 지연을 고정값으로 재현해서(LLM 1회 0.4초, MCP 1회 0.05초 등, 실측 비율을 유지한 축소값), 같은 입력에 대해 **개선 전 동작과 개선 후 동작**을 번갈아 실행해 시간을 잰다. LLM 키나 Docker 없이 누구나 재현할 수 있다. 반복 3회 중앙값. 전체 표는 2-7, 원본은 `reports/bench_speed.md`. 실행: `cd backend && uv run python -m harness.eval.bench_speed --repeat 3 --out ../reports/bench_speed.md`

2-1. 파일 단위 ReAct 조사 병렬화 (B1)

- **문제**: 정적 후보가 있는 파일과 위험 파일 상위 8개를 for 루프로 **하나씩** 조사했다. 파일 하나에 LLM이 최대 8번 호출되므로 감사 시간이 "파일 수 × 파일당 시간"으로 늘어났다. 실측 A/B에서 B 구성(harness)이 84.6초로 단일 프롬프트 방식(23초)의 3.6배였던 주원인이다.
- **해결**: 파일별 조사를 `asyncio.Semaphore(react.concurrency=4)` + `gather`로 동시에 실행한다. 결과는 파일 순서대로 합쳐서 출력이 결정적(deterministic)이다. 감사당 예산(`Budget`)은 공유 객체라서 병렬로 실행돼도 상한이 유지되고, 사용량은 파일별로 따로 모은 뒤 합친다(1-6과 연결).
- **수치**: 파일 10개(파일당 LLM 2회 + MCP 1회) 기준 **8.61초 → 2.58초 (-70%, 3.3배)**. 이론값은 ⌈10/4⌉=3 라운드이고, 동시 실행 수를 늘릴수록 LLM 제공자의 rate limit까지 선형에 가깝게 줄어든다.
- **검증**: 동시 실행 상한(3) 준수, 결과 순서 보존, 사용량 누락 없음(16건)을 확인하는 단위 테스트. 파일별 스크립트로 바꾼 기존 C19·C20 탐지 테스트 포함 ReAct 통합 테스트 11개 통과.
- 파일: `backend/core/graph/nodes/react_investigator.py`, `config/budget.yaml`

2-2. MCP 세션 재사용 + Finding 동시 저장 (B2)

- **문제**: 워커가 MCP 도구를 호출할 때마다 `MultiServerMCPClient`와 세션을 새로 만들었다. 호출 1번에 HTTP 요청이 4건(세션 생성·initialize·`list_tools`·실제 호출) 나갔다. ReAct 스텝, 근거 재확인, Finding 저장처럼 감사 1건에 수십~수백 번 반복되는 호출 모두에 이 비용이 붙었다. correlate 단계는 Finding을 1건씩 순서대로 저장했다.

- **원인 분석**: ① 맥락 헤더(X-Actor-*, X-Audit-Id)가 호출마다 달라질 수 있다고 보고 세션을 호출 단위로 만들었지만, 감사 1건 안에서는 헤더가 항상 같다. ② MCP 클라이언트(`ClientSession.call_tool`)는 출력 스키마 캐시에 도구가 없으면 매번 `list_tools`를 부른다. 새 세션은 캐시가 비어 있으므로 호출마다 목록 조회가 추가됐다.
- **해결**:
  - `(서버, 맥락 헤더)` 별로 세션을 열어 두고 감사가 끝나면 닫는 `audit_gateway()`를 추가했다. API의 1회성 호출은 기존 방식(`http_gateway()`)을 그대로 쓴다.
  - anyio cancel scope는 연 task에서 닫아야 하기 때문에, 세션은 **전용 task가 열고 닫는다**. 다른 task(병렬 ReAct 등)는 열린 세션에 `call_tool`을 동시에 보낸다(요청 id로 다중화).
  - 세션을 열 때 `list_tools`를 한 번만 불러 스키마 캐시를 채운다.
  - 세션이 끊기면 그 세션을 버리고 retryable 오류로 돌려준다. 그러면 기존 재시도(H7 `with_retries`)가 새 세션으로 다시 호출한다.
  - correlate는 구성 Finding을 동시에 저장(최대 8개)한다. `create_finding`이 fingerprint 기준 `ON CONFLICT DO NOTHING`이라 동시 저장해도 멱등이다. 결합 위험 Finding은 구성 ID를 치환해야 하므로 그 뒤에 순서대로 저장한다.
- **수치** (실제 streamable-http MCP 서버를 띄워 측정, HTTP 왕복 5ms 재현):

시나리오	개선 전	개선 후	단축
MCP 호출 60회 순차 (ReAct 파일 내부와 같은 패턴)	2.33초 · HTTP 240건	0.53초 · HTTP 63건	**-77% (4.4배)**
MCP 호출 60회 동시	0.39초 · HTTP 240건	0.14초 · HTTP 63건	-63% (2.7배)
correlate Finding 30건 저장	1.54초	0.21초	**-87% (7.5배)**

- **검증**: 실제 MCP 서버를 상대로 동시 호출 10건이 세션 1개로 처리되는지, 헤더가 다르면 세션이 분리되는지, 끊긴 세션이 복구되는지, 종료 시 세션이 정리되는지 확인하는 테스트. 그래프·공격 경로·업로드 감사 통합 테스트 29개 통과.
- 파일: `backend/core/mcp_client.py`, `backend/core/graph/nodes/correlate.py`, `backend/workers/audit.py`, `backend/harness/eval/runner.py`

2-3. 진행 이벤트 비동기 일괄 기록 (B3)

- **문제**: 진행 이벤트(`investigate.step`, `probe.progress` 등) 하나를 낼 때마다 Redis 명령 5회(Pub/Sub·Stream·expire·status Hash·expire)와 DB 트랜잭션 1회를 **순서대로 await**했다. 이벤트가 많은 ReAct·probe 단계에서는 노드 실행이 매번 이 왕복을 기다렸다.
- **해결**: `buffered_events()` 컨텍스트를 추가했다. 감사 실행 중의 `emit`은 이벤트를 큐에 넣고 바로 돌아오고, 백그라운드 task 하나가 쌓인 이벤트를 최대 200건씩 묶어 **Redis pipeline 1회 + DB `add_all` 트랜잭션 1회**로 기록한다.
  - ContextVar를 쓰기 때문에 병렬 ReAct 같은 자식 task의 이벤트도 같은 큐로 들어간다.
  - 기록하는 task가 하나라서 SSE 재생 순서가 emit 순서와 같다.
  - 블록을 나갈 때 남은 이벤트를 모두 기록한다. 종료 중에 들어온 이벤트는 직접 기록 경로로 보내 유실을 막았다.
  - 버퍼 밖(API 등)에서는 예전처럼 즉시 기록한다.
  - 실패해도 감사를 멈추지 않고 경고 로그만 남기는 기존 정책은 유지했다.
- **수치**: 이벤트 200건(이벤트 사이 노드 작업 1ms, Redis 1ms·DB 4ms 재현) 기준 **2.34초 → 0.24초 (-90%, 9.6배)**. 개선 전 수치는 이전 emit 구현을 그대로 재현해서 측정했고, 개선 후 수치는 마지막 이벤트가 기록될 때까지 포함한 시간이다.
- **검증**: 50건 순차 + 자식 task 3건의 기록 순서·누락 없음, emit이 기록을 기다리지 않음, Redis Stream·status 반영, 블록 밖 즉시 기록을 확인하는 테스트. 워커·그래프·업로드·e2e 테스트 13개 통과.

- 파일: `backend/harness/events.py`, `backend/workers/audit.py`

2-4. 레드팀 probe의 LLM 판정 병렬화 (B4)

- **문제**: 결정론 판정(카나리 유출·금지 도구·외부 도메인)으로 결론이 나지 않은 시도는 LLM이 판정한다. 그런데 probe 하나 안의 시도들, 그리고 probe끼리도 **모두 순서대로** 판정해서 "판정 불가 시도 수 × LLM 지연"만큼 시간이 쌓였다.
- **해결**: 판정 불가 시도를 모아 `asyncio.Semaphore(probe.judge_concurrency=4)`로 동시에 판정한다. probe 안의 시도와 probe끼리 모두 병렬이고, 전체 동시 실행 수는 세마포어 하나로 제한한다. 판정 결과를 반영하고 저장하는 단계(`finalize_probe_verdict`, 성공 횟수·ASR 갱신)는 **시도 순서대로** 진행해서 집계가 항상 같게 나온다. 실패한 판정은 기존처럼 다음 라운드 재시도 대상으로 남긴다.
- **수치**: 판정 불가 시도 18건(probe 6개 × 3회, LLM 0.4초) 기준 **7.26초 → 2.02초 (-72%, 3.6배)**.
- **검증**: 동시 실행 상한(2) 준수, 시도마다 성공 횟수·ASR·Finding이 정확히 반영되는지 확인하는 단위 테스트. 기존 probe_judge·그래프 테스트 통과.
- 파일: `backend/core/graph/nodes/probe_judge.py`, `config/budget.yaml`

2-5. 레드팀 probe 시도 병렬화 (B5)

- **문제**: probe 하나를 3번 시도할 때(ASR 계산용) 대상 에이전트와의 대화를 **순서대로** 실행했다. 대상 에이전트는 응답마다 자체 LLM을 호출하므로 1턴에 수백 ms~수 초가 걸리고, 이 시간이 시도 수만큼 그대로 쌓였다.
- **안전성 확인**: 시도마다 `session_id`가 다르고(`probe-{id}-{uuid}`), 대상 에이전트는 응답마다 그 세션의 도구 호출 기록만 돌려준다. 따라서 시도끼리 대화 기록이나 판정 신호가 섞이지 않는다.
- **해결**: 시도를 `asyncio.Semaphore(probe.attempt_concurrency=3)`로 동시에 실행하고, 결과와 DB 기록은 **시도 번호 순서대로** 합친다. 대상 에이전트가 감당할 수 있도록 동시 실행 수는 설정값으로 둔다. probe끼리 동시 실행(최대 3)은 기존 그대로라서, 대상이 받는 최대 동시 요청은 9이다.
- **수치**: probe 1개 × 시도 3회(대상 응답 0.6초/턴) 기준 **1.81초 → 0.61초 (-66%, 3.0배)**. 기본 probe 12개 × 3회 전체로 보면 대상 에이전트와 대화하는 시간이 이론상 약 1/3로 줄어든다.
- **검증**: 시도마다 응답 지연을 다르게 줘도 0.25초 안에 끝나고(순차면 0.3초), 결과가 시도 번호 순서로 나오는지 확인하는 단위 테스트. business-mcp·업로드 에이전트 테스트 33개 통과.
- 파일: `backend/mcp_servers/business/tools/execute.py`, `config/budget.yaml`

2-6. 워커 1개가 감사를 동시에 여러 건 처리 (B6)

- **문제**: audit-worker는 메시지를 1건씩 받아서 처리가 끝나야 다음 메시지를 받았다. 토픽 파티션이 3개여도 워커가 1개면 감사 요청이 동시에 들어올 때 줄을 섰다.
- **해결**: `consume_loop()`가 **서로 다른 파티션의 메시지를 최대 `AUDIT_WORKER_CONCURRENCY`(기본 3)개까지 동시에** 처리한다.
  - 같은 파티션 안에서는 처리 중인 파티션을 pause하므로 순서가 보장된다.
  - 커밋은 파티션별로 해당 offset만 한다(`{tp: offset+1}`). 그래서 at-least-once와 멱등 키(`kafka:processed:*`)가 그대로 유지된다.
  - 동시에 도는 감사끼리 섞이지 않는지 확인했다. 감사마다 실행 의존성(Deps)·예산·이벤트 버퍼가 따로 있고, trace id는 ContextVar라서 task별로 분리된다. 같은 대상에 대한 동시 감사는 기존 대상 락(1-2)이 막는다.
  - eval-worker는 계속 1건씩 처리한다(기본값 1).
- **수치**: 감사 3건이 파티션 3개에 동시에 도착(감사 1건 1초) 기준 **3.03초 → 1.01초 (-67%, 3.0배)**. 컨테이너를 늘리지 않고 처리량이 파티션 수만큼 늘어난다.

- **검증**: 파티션 3개 × 2건, 동시 처리 상한 2에서 "상한 준수, 파티션 안 순서 유지, 전부 커밋(유실·중복 없음), 처리 중 poll 계속"을 확인하는 테스트와, 상한 1이면 순차 처리되는지 확인하는 테스트. 재시도·DLQ 테스트 통과.
- 파일: `backend/core/kafka/consumer.py`, `backend/workers/main.py`, `backend/core/settings.py`, `.env.example`

2-6b. trace span 묶음 기록 (B7, 진행 중 발견)

- **문제**: 노드 측정(EV4)을 위해 관측성 코드를 보다가, 노드·LLM·MCP 도구 호출마다 생기는 trace span도 **호출마다 DB 트랜잭션을 순서대로 기다린다**는 것을 발견했다. B3와 같은 종류의 병목이고, 감사 1건에 수백 건이 생긴다.
- **해결**: 감사 실행 중(B3의 `buffered_events()` 블록 안)에는 span 행을 이벤트 버퍼에 싣고, 다음 이벤트 묶음과 **같은 트랜잭션으로** insert한다. 보고서처럼 span을 바로 읽어야 하는 곳은 `flush()`로 먼저 기록하고, 블록을 나갈 때 남은 행을 모두 기록한다. 버퍼 밖에서는 예전처럼 즉시 기록한다.
- **수치**: span 300건(DB 트랜잭션 4ms 재현) 기준 **1.92초 → 0.43초 (-78%, 4.5배)**.
- **검증**: span이 감사 실행을 막지 않고, flush·종료 시 빠짐없이 기록되는지 확인하는 테스트.
- 파일: `backend/harness/observability.py`, `backend/harness/events.py`

2-7. 속도 개선 종합

항목	개선 내용	작업량 (재현 조건)	개선 전	개선 후	단축
B1	파일 단위 ReAct 병렬 조사	파일 10개 × (LLM 2회 + MCP 1회)	8.66초	2.59초	**-70% (3.3배)**
B2-a	MCP 세션 재사용 (순차)	MCP 호출 60회, HTTP 왕복 5ms	2.61초 · HTTP 240건	0.62초 · HTTP 63건	**-76% (4.2배)**
B2-b	MCP 세션 재사용 (동시)	MCP 호출 60회	0.54초	0.20초	-63% (2.7배)
B2-c	correlate Finding 동시 저장	Finding 30건	1.56초	0.21초	**-87% (7.4배)**
B3	이벤트 비동기 일괄 기록	이벤트 200건	2.57초	0.27초	**-89% (9.4배)**
B4	probe LLM 판정 병렬화	판정 불가 시도 18건	7.26초	2.02초	-72% (3.6배)
B5	probe 시도 병렬화	probe 1개 × 3회	1.81초	0.61초	-66% (3.0배)
B6	워커 감사 동시 처리	감사 3건 동시 도착	3.03초	1.01초	-67% (3.0배)
B7	trace span 묶음 기록	span 300건	1.93초	0.43초	**-78% (4.5배)**

> 재현 지연값: LLM 0.4초/회, MCP 0.05초/회, Redis 1ms, DB 4ms, 대상 에이전트 0.6초/턴, HTTP 왕복 5ms. 값은 축소했지만 개선 전후는 같은 조건에서 측정했다. B2는 실제 streamable-http MCP 서버를 상대로 측정했다. 위 표는 최종 실행 (`reports/bench_speed.md`, 3회 중앙값) 값이다. 항목별 절의 수치는 개발 중 측정값이라 소수점 둘째 자리가 조금 다를 수 있다. 실제 LLM 환경에서 감사 1건 전체가 얼마나 빨라지는지는 EV5에서 측정한다.

3. 탐지 범위·정확도

3-1. LLM이 규칙에 없는 유형의 취약점을 보고하지 못하던 문제 (S1)

- **문제**: ReAct 조사가 새로 발견한 취약점의 카테고리는 **정적 규칙에 있는 카테고리만** 허용됐다 (`allowed_categories()`). 규칙이 Spring 위주였기 때문에, LLM이 Python 명령 주입·역직렬화·SSRF·XSS 등을 찾아내도 스키마 검증에서 거부되거나 엉뚱한 카테고리로 들어갔다. 규칙이 부족한 부분을 LLM이 메워 주지 못하는 구조였다.
- **해결**: 카테고리 목록을 규칙과 분리했다. `config/categories.yaml`에 10종(COMMAND_INJECTION, INSECURE_DESERIALIZATION, SSRF, XSS, XXE, DEBUG_ENABLED, BROKEN_ACCESS_CONTROL, INSECURE_TRANSPORT, WEAK_CRYPTO, VULNERABLE_DEPENDENCY)을 정의하고, 각각에 설명·기본 심각도·OWASP/CWE 참조를 붙였다. 허용 목록은 "규칙 카테고리 ∪ 설정 카테고리"이고 프롬프트의 카테고리 정의에도 자동으로 들어간다. 새 발견에는 카테고리의 OWASP/CWE 참조가 자동으로 붙는다. 집계·채점의 일관성을 위해 **자유 문자열 카테고리는 계속 거부**한다. 명세와 달라지는 점은 `DECISIONS.md`에 기록했다.
- **검증**: 확장 카테고리 허용 + CWE 참조 부여, 규칙 카테고리 유지, 자유 문자열 거부, 프롬프트 정의 포함을 확인하는 테스트. ReAct·평가 채점 테스트 통과.
- 파일: `config/categories.yaml`, `backend/core/categories.py`, `backend/core/graph/nodes/react_investigator.py`

3-2. 여러 줄에 걸친 코드를 정적 규칙이 놓치던 문제 (S4)

- **문제**: regex 탐지기가 **한 줄 단위**로만 패턴을 검사했다. Spring Security 6의 람다 DSL(`.csrf(c -> c` / `.disable())`), 줄바꿈된 메서드 체인, 여러 줄에 걸친 문자열 결합 SQL처럼 실무 코드에서 흔한 형태를 놓쳤다.
- **해결**: 규칙에 `multiline: true` 옵션을 추가했다. 패턴을 파일 전체에 적용하고, 매치가 시작된 줄 번호로 보고한다(줄당 1건, 시작 줄이 주석이면 제외). 패턴에서 `\s`는 줄바꿈을 넘어가고, 문장 경계를 넘지 않게 하려면 `[^;]*`를 쓴다. 기존 규칙은 기본값(`false`)이라 동작이 바뀌지 않는다.
- **검증**: 여러 줄로 나뉜 CSRF 비활성화 2곳은 탐지하고 주석 처리된 줄은 제외하는지, 같은 규칙을 한 줄 모드로 돌리면 주석 줄만 잡고 실제 2곳은 놓치는지(개선 전 동작) 확인하는 테스트. 기존 규칙 테스트 통과.
- 파일: `backend/core/rules/models.py`, `backend/core/rules/detectors/__init__.py`

3-3. Spring(Java/Kotlin) 정적 규칙 보강 (SP1~SP7)

- **문제**: Spring 규칙이 실무 코드 형태를 많이 놓쳤다.
  - **SQLi**: 같은 줄에서 `jdbcTemplate.query(... "..." +` 형태일 때만 잡았다.
  - **보안 설정 파일 선택**: 파일 이름(`*Config*`, `*Security*`)으로만 골랐다.
  - **CSRF**: 한 줄 패턴만 있었다.
  - **설정 파일**: `application.properties`를 지원하지 않았다.
  - **Kotlin**: 대부분의 규칙이 `.kt`를 보지 않았다.
  - **기타**: 인가 누락, IDOR, XXE, 역직렬화, 명령 주입 규칙이 아예 없었다.
- **해결** (규칙 17개 → 27개, 엔진 기능 3개 추가):
  - **엔진**: 여러 줄 매칭(3-2), `.properties` 파서(yaml_path 탐지기가 `key=value` 처리), `forbid_also`(파일에 방어 코드가 있으면 탐지 안 함), Spring 엔드포인트 분석기(매핑 어노테이션별 메서드 본문을 중괄호 균형으로 잘라 메서드·클래스 인가 어노테이션, 경로 변수 조회, 소유자 확인 여부를 판단).

  - **SQLi** (`SEC-SQLI-002/003`): 변수에 담아 만든 쿼리, `String.format` , `StringBuilder.append` 체인, MyBatis `${}` , 변수로 받은 네이티브 쿼리, Kotlin 문자열 템플릿 SQL.
  - **인가** (`SEC-SPRING-007/009/010`): 민감 경로 `permitAll` (actuator health·info는 정상 설정이라 제외), 민감 경로 엔드포인트의 인가 어노테이션 누락, IDOR 후보(경로 ID로 조회하면서 소유자 확인 없음). 전역 설정과 함께 봐야 하므로 LLM 검토 대상으로 둔다.
  - **CSRF·보안 설정** (`SEC-SPRING-001/002`): 람다 DSL 여러 줄, 메서드 참조 (`AbstractHttpConfigurer::disable`), Kotlin DSL(`csrf { disable() }`). 파일 이름이 아니라 내용 (`SecurityFilterChain` , `HttpSecurity`)으로 대상 파일을 고른다.
  - **기타** (`SEC-SPRING-008/011` , `SEC-JAVA-001~003`): `@CrossOrigin("*")` , 운영 프로필 `show-sql` , 방어 설정 없는 XML 파서(XXE), `ObjectInputStream` · `XMLDecoder` ·XStream·Jackson default typing, 변수로 만든 OS 명령.
- **검증 방법**: 데모 앱을 바꾸면 기존 A/B 기준선과 비교할 수 없게 된다. 그래서 규칙 fixture(`datasets/rule_fixtures/spring/`)에 양성 26건과 음성 11건(클래스 레벨 인가, XXE 방어 설정, 소유자 확인, 바인딩 쿼리, 인자 배열 실행, 주석, actuator health 등)을 두고, **탐지 결과가 기대 목록과 정확히 같은지** 테스트한다 (`test_rule_fixtures.py`).
- **수치** (`harness/eval/rule_coverage.py` , 같은 엔진에 이전 규칙과 새 규칙을 각각 적용):

규칙	fixture 양성 탐지	오탐
이전 (17개)	0 / 26 (0%)	0
개선 후 (27개)	**26 / 26 (100%)**	**0**

> fixture는 이전 규칙의 빈틈(파일 이름 기준 선택·한 줄 매칭· `.properties` ·Kotlin·규칙 없는 유형)을 겨냥해 만들었기 때문에 이전 규칙이 0건이다. 일반적인 코드베이스의 탐지율이 아니라 **보강한 범위**를 보여 주는 수치다.

- **데모 앱 영향**: 새 규칙은 데모 앱에서 `/admin` 엔드포인트 2곳의 인가 누락을 새로 찾았다. 샘플에 원래 있던 실제 취약점인데 정답셋에서 빠져 있었으므로 C21·C22로 추가했다. 정적 규칙 기준선 재현율은 0.90(18/20)에서 **0.909(20/22)**로 올랐다. 나머지 새 규칙은 데모 앱에서 탐지가 없었다(오탐 증가 없음).
- 파일: `rules/code/SEC-SQLI-002·003, SEC-SPRING-007~011, SEC-JAVA-001~003.yaml` (신규), `SEC-SPRING-001·002·004·005·006, SEC-INFO-001, SEC-SECRET-003, SEC-SQLI-001, SEC-PATH-001, SEC-LOG-001, SEC-SPRING-003.yaml` (수정), `backend/core/rules/{models.py, detectors/__init__.py}` , `datasets/rule_fixtures/spring/` , `datasets/ground_truth/code.yaml`

3-4. Python 코드 분석 추가 (PY1~PY3)

- **문제**: SQLi·경로 조작·로그 규칙이 모두 `**/*.java` 에만 적용됐다. Python 파일에 걸리는 코드 규칙은 알려진 키 형식 (`sk-` , `AKIA` 등)과 민감 파일 존재 여부뿐이었다. 엔트로피 기반 비밀 탐지는 큰따옴표 문자열만 봐서, 작은따옴표를 주로 쓰는 Python의 `API_KEY = '...'` 도 놓쳤다. `eval` · `subprocess(shell=True)` · `pickle` ·SSRF·디버그 모드 같은 Python 고유 위험은 규칙이 하나도 없었다.
- **해결**:
  - **PY1 (기존 규칙 확장)**: 엔트로피 규칙이 작은따옴표 문자열도 읽는다. 주석·로그 규칙에 `.py` 를 넣고 `logger.*` · `logging.*` · `print` 패턴을 추가했다.
  - **PY2 (규칙팩 10개, `SEC-PY-001~010`)**:
    - 명령·코드 실행, 역직렬화(pickle·marshal·dill·joblib· `yaml.load` SafeLoader 없이)

        - SQL(f-string·`%`·`.format`·결합, SQLAlchemy `text(f"...")`, Django `raw`/`extra`)
        - SSRF(변수 URL로 보내는 requests·httpx·urllib), 경로 조작(`open(f"...{name}")`, `send_file`)
        - 디버그 모드(Flask `debug=True`, Django `DEBUG`·`ALLOWED_HOSTS=['*']`), XSS(`autoescape=False`, `Markup`, `|safe`, `render_template_string`)
        - TLS 검증 끔, 비밀번호용 약한 해시(md5/sha1)·ECB, lxml XXE
    - **PY3 (AST 탐지기 `ast_python`)**: 명령 실행·역직렬화는 정규식 대신 Python AST로 판정한다.
        - `import subprocess as sp`, `from os import system` 같은 **import 별칭을 원래 이름으로 해석**한다.
        - **여러 줄 호출**을 하나의 호출 노드로 본다.
        - **상수 인자**(`eval("1+1")`, `os.system("clear")`), `shell=True`가 없는 호출, `Loader=SafeLoader`, 메서드 `model.eval()`, `ast.literal_eval`, 문자열·주석 안의 코드는 제외한다.
- **수치**:

측정	개선 전	개선 후
Python fixture 양성 탐지 (`rule_coverage.py`)	0 / 27 (0%)	**27 / 27 (100%)**, 오탐 0
명령 실행·역직렬화 케이스: 단순 정규식 vs AST	정규식: 탐지 6/8, **오탐 8** (상수 인자·주석·문자열·`model.eval()` 등)	AST: **탐지 8/8, 오탐 0**

> 정규식이 놓친 2건은 별칭 import(`system(cmd)`)와 여러 줄 호출(`sp.Popen(` 다음 줄의 `shell=True`)이다. 정규식은 같은 파일의 음성 케이스 중 8개를 잘못 탐지했다.

- **검증**: Python fixture(`datasets/rule_fixtures/python/`, 양성 27곳·음성 15건 이상)의 탐지 결과가 기대 목록과 정확히 같은지 확인하는 테스트. 새 Python 규칙은 데모 Spring 앱에서 탐지 0건(교차 오탐 없음).
- 파일: `rules/code/SEC-PY-001~010.yaml`(신규), `SEC-SECRET-002, SEC-COMMENT-001·002, SEC-LOG-001.yaml`(수정), `backend/core/rules/{models.py, detectors/__init__.py}`, `datasets/rule_fixtures/python/`

3-5. 위험 파일을 이름이 아니라 내용으로 선택 (S2·PY4)

- **문제**: ReAct 조사 대상은 "정적 후보가 있는 파일 + 위험 파일 상위 8개"다. 정적 규칙을 피한 취약점(난독화·문자열 분할·규칙 없는 유형)은 후보가 없으므로 **위험 파일로 뽑혀야만** 조사된다. 그런데 위험 파일을 **파일 이름 키워드**(Config·Client·Payment…)와 `.java` 확장자 가산점으로만 골랐다. `utils.py`·`views.py`·`MemberDao.java` 처럼 이름이 평범한 파일은 조사 대상에 들어가지 못했고, Python 파일은 사실상 선택되지 않았다. 실측에서 정적 회피 탐지율이 0.67 ± 0.58로 흔들린 원인으로 봤다.
- **해결**: `core/risk_signals.py` — 내용 신호에 가중치를 매겨 점수를 낸다. 언어 중립적이며, 같은 신호는 파일당 3회까지만 센다.
    - **진입점·입력**: Spring 매핑, Flask/FastAPI 라우트, `request.*`
    - **보안 설정**: `SecurityFilterChain`, `ALLOWED_HOSTS`, CORS
    - **위험 싱크**: SQL·명령 실행·역직렬화·XML 파서·네트워크·파일
    - **암호·비밀 흔적**: 키 이름에 문자열 대입
    - **난독화**: Base64 디코드, `"sk_live_" +` 같은 키 접두어 분할

    - **이름**: 동점을 가르는 작은 보너스로만 쓴다. 최소 점수는 **내용 점수로만** 판정해서 이름만 그럴듯한 빈 파일은 제외한다. 테스트 경로와 비코드 파일도 제외한다.
    - **연결**: business-mcp에 읽기 전용 도구 `rank_risky_files`를 추가했다(파일 내용은 돌려주지 않음, H3 게이트 intake 허용). 도구가 실패하면 이전 방식(이름)으로 계속하고, 어떤 방식으로 골랐는지 `plan.risky_selected_by`에 남긴다.
- **수치** (`harness/eval/risky_selection.py`, 위험 파일 상위 8개에 취약 코드 파일이 들어간 수):

대상	취약 코드 파일	이름 기준 (이전)	내용 기준 (개선)
데모 앱 정적 회피 파일 (C19·C20)	2	2	**2 (유지)**
Python fixture	5	**0**	**5**
Spring fixture	7	2	**6**

> Spring fixture에서 빠진 1개(`UserRepo.kt`, 점수 5)는 10개 파일 중 9위다. 상위 8개에 든 다른 파일도 보안 설정·진입점이 있어 조사할 가치가 있는 파일이다. 개발 중 C20 파일(`"sk_live_" + ...`)이 선택에서 빠지는 문제를 발견해 분할 키 신호를 고쳤고, 그 결과가 위 2/2이다.

- **검증**: 이름이 평범한 위험 파일은 선택되고 이름만 위험해 보이는 빈 파일·테스트·비코드 파일은 제외되는지, 위 선택률이 유지되는지, 도구 등록·게이트·readOnly 어노테이션을 확인하는 테스트 3개. 그래프·업로드·워커 통합 테스트 통과.
- 파일: `backend/core/risk_signals.py`(신규), `backend/mcp_servers/business/tools/query.py`, `backend/harness/tools.py`, `backend/core/graph/nodes/intake.py`, `backend/harness/eval/risky_selection.py`(신규)

3-6. 업로드한 Python 에이전트의 도구를 놓치던 문제 (PY5)

- **문제**: 매니페스트 없이 업로드한 Python 에이전트는 소스에서 도구를 찾는데, `TOOLS = [...]` 처럼 **리터럴 목록으로 선언한 도구만** 찾았다. 실무에서 흔한 `@tool`(LangChain), `@mcp.tool()`(MCP 서버), `@function_tool`(OpenAI Agents SDK), `Tool(...)`, `StructuredTool.from_function(...)` 방식은 하나도 찾지 못했다. 이런 에이전트는 "도구 없음"으로 처리되어 도구 권한 점검(A1~A4)과 레드팀 계획에서 빠졌다.
- **해결**: AST로 데코레이터가 붙은 함수(`tool`·`function_tool`·`kernel_function`, `mcp.tool`·`server.tool` 같은 속성 형태 포함)와 도구 생성자 호출을 찾는다. 이름은 데코레이터 인자·`name=`·함수 이름 순서로 정하고, 설명은 `description=` 또는 docstring, 파라미터는 함수 인자에서 가져온다(`self`·`ctx` 제외). 코드는 실행하지 않는다(정적 분석만). 어떤 방식으로 찾았는지 `discovered_via`에 남기고, 기존 리터럴 탐색과 결과 형식은 그대로다.
- **수치**: 7가지 등록 방식이 섞인 샘플 에이전트 기준 **찾은 도구 1/7 → 7/7**. 리터럴 목록이 없는 에이전트는 이전 코드에서는 **에이전트로 인식조차 되지 않았다**(`discover_agent` → None). 무관한 데코레이터(`@functools.lru_cache`)는 도구로 잡지 않는다.
- **검증**: 위 7가지 방식과 음성 1건을 확인하는 테스트. 기존 업로드 에이전트 테스트 통과.
- 파일: `backend/core/agent_project.py`

3-7. 파일을 넘나드는 입력→싱크 흐름 추적 (S3)

- **문제**: ReAct 조사가 **파일 단위**로 진행돼서, 입력이 컨트롤러로 들어와 서비스·리포지토리를 거쳐 SQL 같은 싱크에 도달하는 흐름을 한 번에 보지 못했다. 모델이 다른 파일을 보려면 `search_repo`로 직접 찾아야 했다. 또 조사 중인 파일이 아닌 다른 파일에 새 발견을 보고하면 무조건 버려졌다.
- **해결**:

- `core/code_graph.py` : 가벼운 정적 관계 색인을 만든다. Java/Kotlin은 `package` · `import` ·같은 패키지의 타입 이름 참조, Python은 `import` · `from-import` (상대 import 포함)을 본다. 파일마다 "이 파일이 쓰는 파일(uses)"과 "이 파일을 쓰는 파일(used_by)"을 계산한다.
- business-mcp에 읽기 전용 도구 `related_files` 를 추가했다(경로 탈출 검사 포함). ReAct는 파일 조사를 시작할 때 관련 파일 목록을 받는다.
- 프롬프트 v4는 관련 파일 목록을 보여 주고, 입력이 검증 없이 싱크까지 가는지 관련 파일을 읽어 확인하라고 지시한다.
- **관련 파일에 보고한 새 발견은 유지**하고 관련 없는 파일에 보고한 것은 계속 버린다. 관련 파일에 보고한 발견도 다음 단계의 근거 검증(H4)을 거친다.
- 허용 경로 목록(`related_paths`)은 **시스템이 채우고 LLM 출력 스키마에서는 숨긴다**. 감사 대상에 심긴 지시 때문에 모델이 이 목록을 조작해 아무 파일에나 보고하는 일을 막기 위해서다(테스트로 확인).

- **수치** (데모 앱, 정답셋 취약점이 있는 하위 계층 파일 기준):

취약 파일	정답	진입점에서의 연결
OrderQueryRepository	C12 (SQLi)	OrderController → **1단계**
MemberSearchRepository	C13 (SQLi)	AdminController → MemberService → **2단계**
AuthService	C14 (로그)	연결 없음 (샘플에 호출하는 컨트롤러가 없다)

이전에는 관계 정보가 없어서 0/3이었고(모델이 직접 검색), 개선 후 2/3이 관련 파일로 바로 제시된다. 실제 LLM 감사에서 탐지율이 얼마나 오르는지는 EV5(A/B 재측정)로 확인한다. - **검증**: 컨트롤러→리포지토리 관계 반환, 관련 파일 보고는 유지·무관한 파일은 제거, 모델이 끼워 넣은 `related_paths` 는 무시되는지 확인하는 통합 테스트 2개. ReAct 통합 테스트 13개 통과. - 파일: `backend/core/code_graph.py` (신규), `backend/mcp_servers/business/tools/query.py` , `backend/harness/tools.py` , `backend/core/graph/nodes/react_investigator.py` , `backend/harness/prompts/code_investigate_v4.yaml` (신규)

4. 외부 도구 연동

4-1. 외부 SAST·SCA 연동 (E1~E3)

- **문제**: 탐지가 직접 만든 규칙에만 의존했고, **의존성 취약점(SCA)은 아예 검사하지 않았다.** Log4Shell·Spring4Shell처럼 코드가 아니라 라이브러리 버전에서 오는 고위험 취약점을 놓쳤다.
- **해결**: `core/external_scanners.py` + business-mcp 실행 도구 `run_external_scan` (커밋 기준 캐시)
  - **Bandit (Python)**: 런타임 의존성으로 추가했다. test id를 우리 카테고리로 매핑하고, **심각도·신뢰도 MEDIUM 이상**만 받는다. import 경고·LOW 등급 같은 잡음은 후보로 넣지 않는다.
  - **Semgrep**: 설치된 경우에만 실행한다(`--metrics=off` , 규칙 세트는 `SEMGREP_CONFIG`). 결과의 CWE를 카테고리로 매핑한다.
  - **SCA**: `requirements*.txt` · `pyproject.toml` · `pom.xml` (`${property}` 해석)· `build.gradle` 에서 고정 버전을 읽어, 오프라인 로컬 권고 DB(`datasets/advisories/` , Log4Shell·Spring4Shell·Text4Shell·PyYAML 등 대표 CVE 16건)와 OSV 형식 범위로 비교한다. OSV 온라인 조회는 **감사 대상의 패키지 정보가 외부로 나가므로 기본 꺼짐**이다.
  - **통합**: 외부 결과는 정적 후보(NEEDS_REVIEW)로 들어가고 기존 ReAct·근거 검증이 판정한다. 내장 규칙이 같은 위치·카테고리(±2줄)를 이미 잡았으면 버린다. 도구가 없거나 실패·타임아웃이 나도 감사는 계속되고, 상태(`COMPLETED /`

`NOT_INSTALLED / DISABLED / ERROR / TIMEOUT`)가 진행 이벤트에 남는다. 외부 프로세스는 180초 타임아웃이 있고 snippet은 마스킹한다.

- **수치**:

측정	결과
SCA fixture 취약 의존성 (Maven 3·PyPI 3)	개선 전 **0/6** (검사 없음) → **6/6**, 안전 버전·버전 미지정 4건 오탐 **0**
데모 앱 (Boot BOM이 버전 관리, 고정 버전 없음)	SCA 탐지 0건 (오탐 없음)
Bandit 단독, Python fixture (양성 27곳)	매핑 후 18건 중 15건이 양성과 일치, 실제 오탐 2건(`eval("1 + 1")`, 체크섬용 md5), 양성 12건 미탐

> Bandit 단독 결과는 외부 도구를 **보조 신호**로 두고 최종 판정은 ReAct·근거 검증이 하는 구조가 맞다는 근거다. 같은 fixture에서 내장 규칙(AST 포함)은 27/27, 오탐 0이다.

- **개발 중 발견해 고친 버그**: ① Bandit은 `./app/db.py`처럼 실행 디렉터리 기준 상대 경로를 돌려주는데, 이를 워커의 현재 디렉터리 기준으로 해석해서 **결과가 전부 버려졌다**(0건). ② `pom.xml`의 줄 번호가 version 줄이 아니라 artifactId 줄을 가리켰다(정규식 그룹 오프셋 오류).
- **검증**: SCA fixture 정확 일치(6건, 대소문자·속성 해석 포함), 버전 범위 경계값, Bandit 실제 실행(매핑·잡음 제외·경로), Semgrep JSON 매핑, 도구가 없을 때 상태만 보고, 내장 규칙과의 중복 제거를 확인하는 테스트 6개.
- 파일: `backend/core/external_scanners.py`(신규), `datasets/advisories/advisories.yaml`(신규), `datasets/sca_fixture/`(신규), `backend/mcp_servers/business/tools/execute.py`, `backend/core/graph/nodes/code_static_scan.py`, `backend/harness/tools.py`, `backend/core/mcp_client.py`, `backend/core/settings.py`, `backend/pyproject.toml`(bandit), `.env.example`

5. 평가 체계

5-1. Python 평가 대상과 정답셋 (EV1)

- **문제**: 정답셋이 직접 만든 Spring 데모 앱 하나뿐이어서, Python 코드에 대한 탐지 성능은 한 번도 측정된 적이 없었다.
- **해결**: `targets/vuln-python-app`(Flask 형태, 모든 키는 가짜 값)과 정답셋 `datasets/ground_truth/code_python.yaml`을 만들었다.
  - **취약점 16건**: SQLi(입력은 views, SQL 생성은 db — 파일을 넘나듦), 명령 주입, SSRF, 경로 조작, XSS, 민감정보 로그, 인가 누락(관리자 환불), pickle·`yaml.load`, 약한 해시, TLS 검증 끔, 디버그 모드, 하드코딩 키, Base64로 숨긴 키·문자열 분할 키(**정적 회피 2건**), 취약 의존성
  - **함정 6건**: 테스트 토큰, 상수 명령, 체크섬 md5, 바인딩 쿼리, `safe_load`, 환경 변수 비밀번호
  - **감사기 인젝션 2건**: 명령 주입 바로 위의 "보고하지 말 것" 주석, README
  - 위치 검증 스크립트가 `code*.yaml` 전부를 검사하고, seed·demo-reset에도 연결했다.
- **이 과정에서 고친 규칙 빈틈 3건**: ① 큰따옴표 f-string 안의 작은따옴표(`'%{kw}%'`) 때문에 SQL 패턴이 끊김, ② `render_template_string("<h1>" + 입력)` 결합 XSS, ③ Python `tests/` 경로를 테스트 코드로 인식하지 못함(Java `src/test/`만 인식) — ReAct 기각 보류 판정에도 같은 정의를 쓰도록 통일.

5-2. 대상별 코드 감사 평가 도구 + 채점 버그 수정 (EV2)

- **해결**: `harness/eval/code_target_eval.py` (`make eval-target`) — 대상과 정답셋을 지정해 세 가지 구성으로 실행하고 채점한다.
  - **S**: 내장 정적 규칙
  - **SX**: S + 외부 SAST·SCA
  - **H**: 코드 감사 서브그래프 전체 — 정적 → 외부 → ReAct(LLM) → 근거 검증. 운영과 같은 노드·도구·프롬프트를 in-process MCP로 실행한다.
  - 원본 대상은 임시 디렉터리에 복사해 git 초기화하므로 바뀌지 않는다. 정적 회피 재현율, 감사기 인젝션 견고성, 토큰·비용·지연도 함께 낸다. H 구성은 LLM 인증이 실패하면 **정적 판정 수치를 H 결과로 내지 않고 실패로 끝낸다**(수치 조작 방지).
- **채점 버그 2건 수정**: ① INFO 허용 함정(테스트 경로 값)에 INFO로 보고한 것을 일반 오탐으로 셌다(명세 의도와 반대). ② 경량 채점기(`code_eval`)가 함정 오탐을 "함정 오탐 + 기타 오탐"으로 **두 번** 셌다(전체 러너용 `scorer.py` 는 맞게 계산하고 있었다). 두 채점기를 같은 규칙으로 맞췄다.
- **수치** (정적 기준선, 같은 엔진·채점기에서 규칙만 교체):

대상	이전 규칙	개선 후 규칙 (S)	개선 후 + 외부 도구 (SX)
Spring (정답 22건)	재현율 0.818 · 정밀도 0.857 · **F1 0.837**	재현율 0.909 · 정밀도 0.870 · **F1 0.889**	0.909 · 0.870 · 0.889
Python (정답 16건)	**탐지 0건**	재현율 0.75 · 정밀도 1.0 · **F1 0.857**	재현율 0.8125 · 정밀도 0.929 · F1 0.867

> 두 대상 모두 감사기 인젝션 탐지·견고성 1.00이다. 정적 회피 케이스(Spring C19·C20, Python PY13·PY14)는 정적 구성에서 0이며 H 구성(LLM)의 몫이다. SX에서 Bandit은 취약 의존성(PY16)을 더 찾았지만 체크섬 md5 함정(PD03)도 잡았다. 외부 도구 결과를 그대로 확정하지 않고 LLM 검토를 거치게 한 이유다.

- 결과 파일: `reports/code_eval_vuln-python-app.md` , `reports/code_eval_vuln-spring-app.md`

5-3. 비용 산출 (EV3)

- **문제**: `config/pricing.yaml` 의 가격이 0이어서 비용· `f1_per_dollar` 가 계산되지 않았다.
- **해결**: OpenAI 공식 가격을 넣었다(per 1M tokens: gpt-5.4-nano 입력 $0.20 / 출력 $1.25, gpt-5.4-mini $0.75 / $4.50, text-embedding-3-small $0.02). 참고로 이전 실측의 B 구성 감사 1건(약 85.9k 토큰)은 이 가격으로 **약 $0.017~$0.107**이다(입력·출력 비율에 따라 다름. 입력 비중이 커서 하한에 가까울 것으로 본다).
- **함께 고친 버그**: `.env` 에만 키를 둔 로컬 실행에서 키가 `ChatOpenAI` 로 전달되지 않았다(Docker에서는 환경 변수로 들어가서 드러나지 않았음). 설정값을 명시적으로 넘긴다.

5-4. 감사 보고서의 노드·도구별 소요 시간 (EV4)

- **문제**: 감사가 어디서 오래 걸리는지 보고서에서 볼 수 없었다. trace span이 DB에 쌓이고 있었지만 집계해서 보여 주지 않았다.
- **해결**: `latency_breakdown()` 이 감사 1건의 span을 노드·LLM·도구별로 집계(호출 수, 합계, 최대 ms, 토큰)한다. 보고서 JSON에는 `timings` , Markdown에는 "소요 시간" 표로 들어간다. 묶음 기록 대기 중인 span도 먼저 flush해서 포함한다(B7과 연결). 이 작업 중에 span 기록 자체가 병목인 것을 발견해 B7로 개선했다.

5-5. 실제 LLM 재측정 (EV5) — 미완료

- `.env` 의 OpenAI 키가 **`invalid_api_key` (401)로 거부**되어 실제 LLM 측정(H 구성, 전체 A/B)을 진행하지 못했다 (2026-11-06 확인).
- 키를 교체한 뒤 아래 명령으로 재측정하고 이 절을 채운다.
  - `make eval-target TARGET=vuln-spring-app TRUTH=code.yaml CONFIGS=S,SX,H REPEAT=3`
  - `make eval-target TARGET=vuln-python-app TRUTH=code_python.yaml CONFIGS=S,SX,H REPEAT=3`
  - `make eval` (전체 A/B, Docker 스택 필요)
- 확인할 목표(이전 실측 기준): 재현율 ≥ 0.9, 감사기 견고성 1.0, 정적 회피 탐지율(이전 B 0.67 ± 0.58), 감사 1건 지연(이전 84.6초)과 토큰(이전 약 86k).

---

6. 코드 리뷰 반영: 동시성·실패 경로

> 외부 코드 리뷰(`codeReview.md`)의 지적 A·B·C를 검증하고, 리뷰가 놓친 문제 2건을 더해 고쳤다. 각 항목은 **수정 전 코드에서 실패하는 테스트**로 확인했다(`tests/unit/test_review_fixes.py` 15개 중 11개가 수정 전 실패).

6-1. 대상 락의 확인-후-삭제 경쟁 (리뷰 A)

- **문제**: 락 해제가 `GET` (내 것인가?) → `DEL` 두 번의 명령이었다. 그 사이 내 락이 만료되고 다른 감사가 새로 잡으면, 남의 락을 지운다(TOCTOU). 그 결과 같은 대상에 감사가 동시에 돈다. 연장(`GET` → `EXPIRE`)과 감사 취소 API도 같은 구조였다.
- **원인**: 원자적 해제 Lua 스크립트(`_RELEASE`)는 선언만 되어 있었다. 연장은 지난 작업(1-2)에서 테스트용 fakeredis가 Lua를 못 돌려 일부러 나눴다. **테스트 편의 때문에 원자성을 포기한 것**이다.
- **해결**: 획득·해제·연장을 Lua 스크립트 한 번으로 처리한다. 연장은 여러 대상 락을 한 번에 처리하고 연장한 개수를 돌려준다. 개수가 줄면 "락 잃음" 경고를 남긴다. 테스트는 `lupa` (개발 의존성)로 fakeredis에서도 Lua를 실제로 실행한다.
- **검증**: A의 락 만료 → B 획득 → A 해제 시도 → **B의 락이 남는다**. 여러 대상 중 내 락만 연장하고, 없는 락은 새로 만들지 않는다.

6-2. Kafka 메시지 lease를 잃어도 계속 처리하던 문제 (리뷰 B + 보완)

- **문제**: heartbeat가 `SET XX` 로 "키가 있으면 덮어쓰기"만 했다. lease가 만료되어 워커 B가 메시지를 가져간 뒤에도 이전 워커 A의 heartbeat가 **B의 lease를 A 것으로 덮어썼다.**
- **리뷰보다 더 나간 부분**
  - 소유자 확인만 넣으면 A는 lease를 잃은 사실을 알 뿐, 감사를 끝까지 실행한다(중복 처리). 그래서 **lease를 잃으면 처리 중인 핸들러를 취소한다.** 실패 라우팅(retry/DLQ)과 `done` 표시도 하지 않는다. 새 소유자가 체크포인트에서 이어 처리한다.
  - 소유자 값이 프로세스 단위(`WORKER_ID`)였다. 같은 워커 안의 두 번째 획득이 "내 것"으로 그냥 통과했다. 워커 1개가 동시에 여러 건을 처리하게 바꾼 뒤(2-6)로는 이것이 실제 위험이 된다. **획득할 때마다 새 토큰**을 발급한다.
  - heartbeat에서 Redis 오류가 나면 heartbeat task가 조용히 죽어 lease가 만료됐다. 그 예외는 정리 단계에서 다시 올라왔다. 이제 경고를 남기고 다음 주기에 재시도한다.

- **검증**: lease를 빼앗긴 핸들러는 끝까지 실행되지 않는다. 새 소유자의 lease 값은 그대로 남는다. 실패로 라우팅되지 않는다. 같은 프로세스의 두 번째 획득은 기다린다. 바깥 취소(종료·리밸런스)는 그대로 전파된다. heartbeat의 일시적 Redis 오류는 처리를 깨지 않는다.

6-3. 병렬 LLM 호출이 예산 상한을 넘던 문제 (리뷰가 놓친 문제)

- **문제**: 예산 확인이 "지금까지 쓴 양 < 상한이면 호출"이었다. 파일 4개를 동시에 조사하면(2-1) 4개가 모두 "아직 남음"을 보고 호출한다. 그래서 최대 4회분까지 상한을 넘었다. **2-1의 속도 개선이 만든 부작용**이다.
- **해결**: 호출 전에 예상 사용량(입력 길이 + 출력 예상치)을 **예약**한다. "쓴 양 + 진행 중 예약 + 이번 예상"이 상한을 넘으면 거부한다. 확인과 예약은 await가 없는 동기 코드라 asyncio에서 끼어들 수 없다. 호출이 끝나면 실제 사용량을 반영한 뒤 예약을 푼다. 실패해도 예약은 풀린다.

상한 10,000 토큰, 1회 2,500 토큰 호출 8개 동시 (실측)	수정 전	수정 후 (출력 예상치 기본 2,000)	수정 후 (예상치 2,500)
실행된 LLM 호출	8회	4회	3회
사용 토큰	**20,000 (상한의 2배)**	10,000 (상한 이내)	7,500

- **한계**: 상한 근처에서는 예약 때문에 예전보다 1회 일찍 멈출 수 있다. 실제 사용량이 예상보다 크면 그 차이만큼은 넘을 수 있다.

6-4. 병렬 작업 하나가 실패하면 나머지가 고아로 남던 문제 (리뷰 C)

- **문제**: `asyncio.gather`는 첫 예외를 바로 올리고, 나머지 task는 **취소하지 않은 채 계속 실행**한다. probe 시도 병렬화(2-5)에서 시도 하나가 실패하면, 나머지 시도가 이미 닫힌 httpx 클라이언트로 계속 요청을 보냈다.
- **해결**: `gather_or_cancel()`은 하나가 실패하면 나머지를 취소하고, 정리될 때까지 기다린 뒤 첫 예외를 올린다. `TaskGroup`은 예외를 `ExceptionGroup`으로 감싸 기존 `except AppError` 처리를 깨뜨리므로 쓰지 않았다. 적용 범위는 이번에 병렬화한 6곳(probe 시도, 파일별 ReAct, probe 실행·판정, Finding 저장, 외부 스캐너)이다.
- **검증**: 시도 1이 실패하면 시도 2·3은 닫힌 클라이언트를 쓰기 전에 취소된다(수정 전 실패). 예외 타입과 결과 순서는 그대로 유지된다.

6-5. 이벤트 묶음 기록의 내구성 트레이드오프 (리뷰가 놓친 문제, 문서화)

- 진행 이벤트·trace span을 묶어서 기록하므로(2-3), **프로세스가 강제 종료되면** 큐에 남은 이벤트와 아직 기록하지 않은 span을 잃는다.
- 정상 종료와 취소(6-2의 lease 상실 포함)에서는 모두 기록된다. 감사 결과(Finding·보고서·체크포인트)는 이 버퍼를 거치지 않는다. 잃는 것은 진행 표시와 관측 데이터뿐이다.
- 속도 개선(-89%)을 위해 이 손실을 받아들였고, 그 판단을 `DECISIONS.md`에 기록했다.

6-6. 리뷰를 검증하며 판단한 것

- 리뷰의 지적 3건은 모두 코드에서 사실로 확인했다. 다만 심각도는 다시 보았다. 락 해제 경쟁은 heartbeat가 TTL을 계속 늘리므로 발생 확률이 매우 낮다. 그래도 고치는 비용이 작아서 고쳤다.
- 리뷰가 칭찬한 "성능을 수치로 검증"은 **고정 지연을 재현한 벤치마크**다. 실제 감사 1건의 지연은 아직 측정하지 못했다(5-5). 이 구분을 유지한다.